LeakLens: Detecting Group and Temporal Data Leakage in Machine Learning Code by Linking Static Analysis to an Instrumented Run

Srijeet Prasad Banerjee **Adrish Sarkar** **Ankur Roy**

Sayandeep Dey **Soumyarik Mukherjee** **Sayan Hazra**

Department of Information Technology, Guru Nanak Institute of Technology, Kolkata, India

Abstract. Data leakage lets information from the evaluation data reach model training, so a reported score no longer predicts performance on new data. Tools that read machine learning code detect preprocessing, overlap and multi-test leakage, but not group leakage (the same patient, user or device on both sides of a split), and they cover temporal leakage only in part, although these are the kinds that distort results most. We present LeakLens, a prototype for Python scripts and notebooks. A static pass based on abstract interpretation reports six leakage types and nominates the split call and the candidate entity or time column; an optional instrumented run checks, fold by fold, whether entities or time periods overlap at that call, with nothing declared by the user; and a located leak can be fixed in a copy of the pipeline and re-run to measure its effect on the pipeline's own metric. On the held-out half of a benchmark of injected leaks (79 files, 73 leaks), scored once under a frozen protocol, the static pass finds 59 leaks and the combined mode 65, with no false positive. Scored per file, the analyser of Yang et al. is right for 20 of the 37 files with a leak of a type it supports, LeakLens for 33 ($p < 0.001$). Given the same exported splits, four data-level checkers and a run-time checker find group and time-order leaks only once the right column is named; without it the best finds 3 of 8 shuffled time-series splits, LeakLens 8 of 8. Only 3 of 7 group findings name the true entity, and a proxy-model tool estimated the impact of group leaks more accurately. All results are on synthetic files written by the authors; real notebooks are not yet labelled.

Keywords: data leakage, static analysis, dynamic analysis, Jupyter notebooks, machine learning engineering, reproducibility

1. Introduction

A machine learning score is meaningful only if the model never saw the evaluation data during training. *Data leakage* is any path by which evaluation rows, or information that is not available at prediction time, influence training [1]. Leaky code runs without error and its tests pass. The only symptom is a score that is better than it should be.

Leakage is common. A survey of earlier reviews found it in 294 papers across 17 scientific fields; in one case study, complex models lost their reported advantage over logistic regression once the errors were corrected [2]. A static analysis of 107,603 public GitHub notebooks reported at least one leak in 29.6% of them [3].

Not all leaks do equal damage. Across 2,047 tabular datasets, fitting a scaler on all rows before splitting, the textbook example, changed AUC by at most 0.005 in all nine conditions tested, while leakage across group and time boundaries was invisible under random cross-validation [4]. In translational EEG research, only 17 of 63 surveyed deep learning studies clearly avoided placing segments of one subject in both training and test data, and in one experiment accuracy fell from 99.8% to 53.0% when subjects were held out properly [5].

The tools that read code look mostly at the first kind. The reference static analyser of Yang et al. [3] and the tools built on it [6, 7, 8] detect preprocessing, overlap and multi-test leakage. To our knowledge, no published static analyser detects group leakage from source code, and temporal leakage is covered in part by one recent LLM-based linter [9]. Tools that check the data itself can find date overlap [13] and group overlap [19, 18], but the user has to export the training and test tables and usually name the column, and the result does not point to a line of code. None of these tools reports how much a detected leak changes the user's own result.

This paper describes LeakLens[1], which addresses these three gaps. Its contributions are:

1. **Group leakage detected from source code**, with the entity inferred from column and variable names and from identifiers derived from file paths. Temporal leakage is detected through data flow. This is not a first for temporal leakage [9]; we treat it as an improvement on existing code-level checks.
2. **A link from static findings to an instrumented run.** The static pass nominates the split call and candidate columns; the run checks entity and time overlap at that call, per fold, with no user declaration.
3. **A per-finding impact estimate** obtained by re-running the user's own pipeline with a canonical fix.
4. **A benchmark of 175 files with 162 injected leaks** that includes group and temporal leakage, with a development and test split fixed before any detection rule was written.

Every number in this paper is measured on synthetic files that we generated; 150 real notebooks are sampled but not yet labelled, so we make no claim about precision or recall on real code.

2. Background and Related Work

Taxonomies. Kaufman et al. [1] formalised leakage. Kapoor and Narayanan [2] describe eight types, among them preprocessing on training and test data together, duplicates, temporal leakage and non-independence between training and test samples. Roth [4] measured severity by type and found that the commonly taught type matters least.

Static analysis of ML code. Yang et al. [3] model data flow in Datalog and report three types, with a precision of 97.6% and a recall of 67.8% on hand-labelled notebooks.

[1] An unrelated package with the same name exists on PyPI. The name of this project may change before any release.

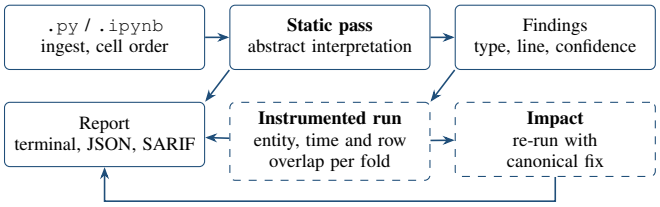


Figure 1. Stages of LeakLens. Dashed stages execute the code and are optional.

NBLyzer and later abstract-interpretation work verify the absence of train/test contamination [10, 11, 12]. LeakageDetector wraps the analyser of Yang et al. in an IDE with quick fixes [6, 7]. Draoui et al. [8] combine static slicing with LLM prompts; their abstract (all we could read) reports F1 gains of 22% for preprocessing and 15% for overlap leakage. scicodelint [9] asks a local LLM pattern questions; two of its patterns concern temporal leakage (shuffled cross-validation on temporal data; forward-looking shifts and centred windows), and none concerns groups.

Checks on data and at run time. Deepchecks [13] checks duplicate rows and date overlap in user-supplied training and test tables, and recent packages add group overlap [19, 18, 20]. splitguard [17] instruments splits and fits and flags group overlap at the split line, for group identifiers the user registers. ArgusEyes [14] detects row overlap through provenance tracking.

Measuring the effect. LeakHound [18] estimates inflation with a proxy model it trains on the given split. mlwhatif [15] re-executes user pipelines with patches, for data-cleaning questions. Leakly [21] re-runs a pipeline on permuted labels; a score above chance suggests leakage somewhere, without locating or fixing it. In LeakLens the estimate belongs to one located finding and is measured on the user's own pipeline and metric. Table 1 compares LeakLens with the analyser of Yang et al., the closest prior work and the engine inside the LeakageDetector tools.

3. Leakage Taxonomy

LeakLens reports six types (Table 2): the three of Yang et al. [3], group and temporal leakage (non-independence and temporal leakage in the taxonomy of Kapoor and Narayanan [2]) and target-derived features (their illegitimate features).

A group finding depends on the claim being made: a model meant for the same patients it was trained on may legitimately share patients across a split. The tool cannot know the claim, so group findings carry a confidence level and can be suppressed inline with a stated reason.

4. Design of LeakLens

Figure 1 shows the three stages. The static stage never runs the analysed code; the other two execute it and are optional.

4.1 Static pass

Notebooks are read in execution order when every code cell has a unique execution count, and in cell order otherwise. One pass of abstract interpretation over the flattened module gives each variable an abstract value: the origin of a data frame, its known columns, which statistics were computed over which rows, whether rows were copied or resampled,

```
recordings = {f"sub-{i:02d}_run-1.npy": ... }
windows, labels, subjects = [], [], []
for fname, signal in recordings.items():
    subject = fname.split("_")[0]
    for w in signal.reshape(-1, 500):
        windows.append([w.mean(), w.std()])
        labels.append(...)
        subjects.append(subject)
X, y = np.array(windows), np.array(labels)
print(cross_val_score(LogisticRegression(), X, y, cv=5).
    mean())

line 16 [high/medium] group (LL-GRP-003): rows split
    without regard
  to `subjects`. fix: use GroupKFold with groups=`
    subjects`
```

Listing 1. Group leakage through cross-validation with the default splitter, and the finding the static pass reports for the full file (code and message shortened).

and the training or evaluation role of a split's result. Rules fire at *sinks*. At a split point (`train_test_split`, a splitter's `split`, a cross-validation helper, a search object, or a manual random sample and its complement) the analyser checks for group and temporal leakage and reports every statistic or copied row that reached the data being split; at a model fit, statistics computed on rows that include the evaluation side; at a prediction or score, whether the model was fitted on the scored rows; and a selection over scores computed in a loop gives a multi-test finding.

The evidence for an entity is a column or variable name, or an identifier derived from a file path (for example a subject code split from a recording's file name). Strong names such as `patient`, `subject` or `user` can give medium confidence; weak names such as `customer`, `id` or `visit` give low confidence only. Listing 1 shows a file-path case.

The analyser prefers missing a leak to inventing one: an unmodelled call depends on all its inputs, and anything unresolved produces no finding. Loops are analysed once, functions inlined to a fixed depth, and classes are opaque. pandas, NumPy, scikit-learn [16] and imbalanced-learn are modelled by name; catalogues and thresholds are in configuration files.

4.2 Instrumented run

Static analysis cannot see an entity column the code never mentions, as when a table with a `patient_id` column is split at random. The dynamic pass runs the script in a child process with scikit-learn entry points wrapped: `train_test_split`, every splitter's `split`, the cross-validation helpers, estimator `fit`, `predict` and `score`, and the metric functions. At each split and fold it records three facts: rows hashed by value (numbers rounded) on both sides; entity columns, looked up in the row-aligned data frames of the script's namespace and reported when their values repeat and enough evaluation rows belong to an entity with training rows; and, for a time column, whether training rows are dated after the earliest evaluation row. These facts are linked to static findings by call site, so a finding ends up *confirmed*, *refuted* or *not checked*; findings only the run can make, such as duplicate raw records, are added.

4.3 Impact estimation

For a located finding, LeakLens re-runs the script with one call site changed. Six fixes are implemented: duplicated records removed before a split; a row-level split made group-

Table 1. Design comparison between the static analyser of Yang et al. and LeakLens. Entries for Yang et al. are taken from their paper and from our own runs of their released tool. The two accuracy rows were measured on different data with different labelling and are not comparable; Table 4 gives the only like-for-like measurement.

Aspect	Yang et al. [3]	LeakLens (this work)
Analysis technique	Static data-flow analysis written in Datalog (Soufflé), with type inference	One-pass abstract interpretation over the Python syntax tree, linked to an optional instrumented run
Runs the analysed code	No	Optional (dynamic and impact modes only)
Input	Python scripts; notebooks have to be converted first	`.py` and `.ipynb` directly, in execution order when it is recorded
Leakage types	Preprocessing, overlap, multi-test	The same three, plus group, temporal and target-derived features
Group and temporal leakage	Not types it claims	Group: entity inferred from names and file paths, confirmed on the data at run time. Temporal: non-chronological splits and forward-looking features
Location reported	The affected evaluation line (as observed in our runs)	The line that causes the leak, with a suggested fix
Confidence and severity	Not reported	Both, per finding
Impact on the score	Not estimated	Re-runs the pipeline with a canonical fix; six kinds of fix so far
Output and integration	HTML report, command line	Terminal, JSON, SARIF; pre-commit hook; GitHub Action
Dependencies	Soufflé, a pyright fork, Python 3.8, Linux	Python 3.12 with NumPy, pandas, scikit-learn
Evaluation data	100 hand-labelled real notebooks (94 analysed) and 107,603 public GitHub notebooks	175 synthetic files with injected leaks, written by us; 150 real notebooks sampled but not yet labelled
Accuracy reported by the authors	Precision 97.6% (40/41), recall 67.8% (40/59), real notebooks	Precision 1.00, recall 0.81 (static) and 0.89 (with the run), own injected TEST split
Time per file	Mean 3.23 s (their paper); median 6.2 s on our files (Kaggle CPU)	Static: median 2.6 ms. Instrumented run: median 1.3 s (laptop)
Large-scale study	29.6% of 107,603 notebooks flagged	None yet

Table 2. The six leakage types and the signal used by each pass.

Type	Static signal	Run-time check
Preprocessing	A transformer, selector or encoder is fitted on rows that later reach the evaluation side	The fit included evaluation rows
Overlap	Resampling or concatenation before the split; evaluation rows flow into a fit	Identical rows on both sides of the split
Multi-test	Several candidates scored on the same evaluation rows feed a selection	Same rows scored repeatedly
Group	A row-level splitter or default CV on data with a candidate entity	The same entity value on both sides
Temporal	A shuffled split on time-ordered data; forward-looking feature	Training rows dated after evaluation rows
Target-derived	A feature computed from the label	Very high single-feature score

aware on the detected column; a non-chronological split made chronological; evaluation rows removed from a model fit; a transformer fitted on all rows refitted on the training rows of a hold-out split; and a feature removed. The estimate is the printed metric of the original run minus that of the fixed run. Findings without an automatic fix, or whose fixed run fails, are reported as "impact unknown", never dropped; if the fixed run takes a different path, the estimate is marked unreliable.

4.4 Running untrusted code, reporting and integration

Notebooks from the internet run only through a sandbox in an offline cloud session: a child process with an allow-listed environment, a throwaway working directory, a process-group timeout and an unprivileged user. Each layer answers a measured problem: the platform (Kaggle) injects session tokens into every job, and a root child could read them from `/proc` even with a clean environment. After the privilege drop, a scan from the child found the token in none of its ancestor processes; a control scan as root found it in two.

Each finding carries the file, line range, type, rule identifier, confidence, severity, evidence, a suggested fix and, when measured, the metric before and after the fix. Output is terminal text, JSON with a published schema, or SARIF, which GitHub code scanning accepts; a pre-commit hook and a GitHub Action are provided. The tool is deterministic and uses no language model. Its 373 tests include property-based tests that the analyser terminates and gives identical output on repeated runs; on a laptop 371 pass and the 2 sandbox tests that need root are skipped (they pass as root in the cloud job).

5. Benchmark and Evaluation Protocol

5.1 Injected benchmark

No public labelled dataset includes group and temporal leakage, so we built one. Thirteen clean base pipelines in three families (tabular, time series and grouped data such as multi-visit patients and EEG-style windows) are mutated by 18 operators. Seventeen inject a leak, for example a scaler fitted before the split, oversampling before the split, a shuffled split on time-ordered data, or a row-level split on grouped data. One produces a *trap*: code that resembles a leak but is correct. Six files combine two leaks. The result is 175 files (94 scripts and 81 notebooks) with 162 labelled leaks, 13 clean files and 6 traps. The lines that carry each leak are known by construction, and every leaky file has a corrected version, so the true inflation of its metric can be measured.

Temporal leaks are divided into three categories: *common* (for example an explicit shuffled split), *differentiating* (cases that pattern-based checks do not cover, such as the default shuffle of `train_test_split`) and *hard* (for example a time column with a name that does not suggest time).

The benchmark is split by base pipeline into DEV (7 pipelines, 96 files, 89 leaks) and TEST (6 pipelines, 79 files, 73 leaks). The assignment was committed before any detection rule existed. Rules and thresholds were tuned on DEV only.

5.2 Real notebooks

We sampled 150 public Kaggle notebooks in three query-based strata of 50 and split them by author into DEV (71) and TEST (79), to be labelled by two people who see no tool output, with Cohen's kappa reported. **This labelling has not been done.** On the 71 DEV notebooks the static pass ran without a crash (median 4 ms per notebook); without labels, what it flagged is not a result.

5.3 Protocol

The protocol was written after DEV results had been seen and frozen before any TEST result existed. A finding matches a label when it is in the same file and cell, its lines overlap the labelled lines, and its type is the labelled type or a listed alternative; matching is one-to-one, an unmatched finding is a false positive and an unmatched label a miss. The operating point is medium or higher confidence. Recall is reported per type with 95% percentile bootstrap intervals over files (2,000 resamples). TEST is scored once per release; any later change is logged and its result labelled a second look. The true inflation of an injected file is the metric of the leaky file minus that of its corrected version, averaged over five data seeds. The generators were designed, and for group leakage adjusted after a first measurement, so that each leak type can change the metric; these values therefore score detection and impact estimates only and are no evidence of severity in practice.

5.4 Baselines

Yang et al. We built the released analyser [3] from source in a Linux cloud job and ran its default command. Its packaged Datalog engine no longer installs, so the engine was built at version 2.4.1, a deviation from the released setup; notebooks were given as plain scripts, as its documentation requires. It reports the affected evaluation line, not the leaking line, so both tools are scored per file and type: a tool is right about a type in a file when it reports that type and the file has a labelled leak of that type. This rule was fixed before any baseline output on TEST existed.

Data-level and run-time tools. leakcheck-ml [19], LeakHound [18], dataleaks [20] and Deepchecks [13] take a training and a test table, not code. The instrumented run of LeakLens exports, at every split call and for up to five folds, the tables the pipeline used (every row-aligned column of the script's data frames, the label column removed and the target kept apart). Each tool received every pair twice, with its defaults and with the true entity or time column named through its own parameter; a detection is placed at the split call's line. splitguard [17] instruments a run itself and was run as documented, and again with the true group declared through `mark_groups`. Each tool ran in its own environment at the cited version (Deepchecks needs Python 3.11 and NumPy 1.26). For every tool a finding is one per file, line and type. These untuned tools ran in jobs after the single TEST run of LeakLens.

Not run. scicode-lint [9] needs a GPU with at least 16 GB and native FP8 support. On two NVIDIA T4 GPUs (no FP8) its pinned server (vLLM 0.18.0) failed to start, and a fallback (llama.cpp, same model at 8 bits) produced no result within 20 minutes for any of the 29 files it attempted; it is reported as not run, not as zero. LeakCheck [8] needs a paid LLM interface and an access-restricted package.

Table 3. Detection on the injected TEST split (79 files, 73 leaks), scored once. Findings of medium or higher confidence. Recall with 95% bootstrap interval over files. No false positive in either mode.

		Static		Static + run	
Type	**Leaks**	**Found**	**Recall**	**Found**	**Recall**
Preprocessing	22	21	0.95 (0.84–1.00)	21	0.95 (0.84–1.00)
Overlap	9	7	0.78 (0.44–1.00)	9	1.00 (1.00–1.00)
Multi-test	6	5	0.83 (0.43–1.00)	5	0.83 (0.43–1.00)
Group	7	3	0.43 (0.00–0.83)	7*	1.00*
Temporal	25	20	0.80 (0.62–0.95)	20	0.80 (0.62–0.95)
Target-derived	4	3	0.75 (0.00–1.00)	3	0.75 (0.00–1.00)
All types	73	59	0.81	65	0.89

*By the matching rule, which checks type and line. Only 3 of the 7 findings name the true entity column, so the strict figure is 0.43.

6. Results

6.1 Detection on the held-out split

Table 3 gives the single TEST run: 59 of 73 leaks found statically (0.81) and 65 with the run (0.89), no false positive in either mode, and no finding on the 9 leak-free files. The run confirmed 43 static findings, refuted none, could not check 25 (no run-time check exists for forward-looking features or pandas statistics) and added 2, both duplicated records. By temporal category the tool found 5 of 5 common, 7 of 7 differentiating and 8 of 13 hard leaks. On DEV, where the rules were tuned, it found 80 of 89 statically and 89 of 89 with the run; the drop from DEV to TEST is real.

6.2 What went wrong on TEST, and a second look

Right line, wrong column. In a patient-visits pipeline the static pass made a low-confidence guess on a column named `visit`; the confirmation step looked only at nominated columns, found visit numbers on both sides of the split, as they always are, and raised the confidence without examining `patient_id`. The matching rule does not check the column, so Table 3 counts these four findings as correct. They are not, and neither is the impact estimate that follows. **A slice mistaken for a split.** A pipeline dropping its last row with `data.iloc[:-1]` was read as the training side of a manual split: five temporal leaks were reported at low confidence only and one multi-test leak was missed. **An unresolved column filter.** A feature list built with a `startswith` filter could not be resolved, so two leaks were not reported.

The three faults were then fixed and logged. DEV was rerun unchanged, and TEST again gave 67 of 73 with the static pass and 73 of 73 with the run, with no false positive; the patient findings now name `patient_id`, and the EEG ones a run-time variable holding the subject identifiers. The fixes were written knowing which TEST files they repair, so this is not a held-out result and Table 3 remains the result of record. The injected benchmark is now saturated on both halves.

Table 4. Yang et al. and LeakLens (static mode) on the 79 injected TEST files, scored per file and type. "Found": files with this leak in which the tool reports the type. "False": files where it reports the type wrongly.

		Yang et al. [3]		LeakLens (static)	
Leakage type	**Files**	**Found**	**False**	**Found**	**False**
Preprocessing	22	10	1	21	0
Overlap / duplicates	9	7	0	7	0
Multi-test	6	3	0	5	0
Group†	7	0	0	3	0
Temporal†	24	3	0	19	0
Target-derived†	4	0	0	3	0

†Not a type the baseline claims to detect. It is credited under temporal in 3 files where it reports whole-series scaling as preprocessing leakage. Temporal has 24 files for 25 leaks because one file holds two.

6.3 Comparison with Yang et al. on the same files

On the three types the baseline supports (Table 4), it found 20 of 37 files with one false alarm, and the static pass of LeakLens 33 of 37 with none; both analysed all 79 files and reported nothing on the leak-free ones. On the 37 file-and-type pairs LeakLens alone was right in 14, the baseline alone in 1, both in 19 and neither in 3 (exact McNemar test, $p = 0.001$). The difference comes from preprocessing (12 against 1); on overlap the tools agree on every file. The comparison favours LeakLens: neither tool was tuned on these files, but they were generated by our operators, with the kinds of leak LeakLens was built for. The figures the baseline's authors report on their own notebooks (precision 97.6%, recall 67.8%) are a different measurement and cannot be set against Table 4.

6.4 Leaks at a split: data-level and run-time tools

Table 5 covers the TEST leaks at a split call, the only ones a data-level tool can see: 8 non-chronological splits of time-ordered data, 7 group splits and 2 sets of duplicated raw records. Without a named column, leakcheck-ml finds 4 of 7 group splits, all with the right column, and 3 of 8 time splits; LeakHound, Deepchecks and splitguard find none of either. With the column named, leakcheck-ml and LeakHound find 7 of 8 and 7 of 7. dataleaks reaches 7 of 7 group splits without a hint, but names an unrelated column in 3 and reports a group overlap in 2 of the 9 leak-free files; its default configuration has no split-order check. Deepchecks has no group check and finds time splits only when told the time column. splitguard finds group splits only when the group is declared, and does not see the held-out rows of a split taken with `next(splitter.split(...))`. All data-level tools find the duplicated records.

LeakLens finds 8 of 8 time splits without being told any column. On group splits it is weaker than the rule-based count suggests: only 3 of its 7 findings name the true entity, against 4 for leakcheck-ml without a hint. Every other tool also needs the user to export the tables, which the instrumented run of LeakLens did for them here.

6.5 Impact estimates

On TEST an estimate could be produced for 34 of 70 located leaks (group 7 of 7, target-derived 3 of 3, overlap 7 of 9, preprocessing 10 of 21, temporal 7 of 25, multi-test 0 of 5); the rest are reported as unknown with the reason. The instrumented script printed exactly the plain script's metric in all 395 runs. For 26 single-leak findings with a bounded metric, averaged over five seeds, the mean absolute error against the true inflation was 0.035 and the rank correlation 0.58. The estimate was exact where our fix is the same operation as the benchmark's corrected file, a weak test, and wrong in direction for hold-out group splits (-0.084 against a true $+0.212$ AUC) because the fix grouped by the wrong column. In the second look the error was 0.0006 and the correlation 0.97, with the same caveat as for detection.

Table 5. Leaks at a split call, injected TEST. Col.: whether the true entity or time column was named to the tool. T1: non-chronological splits of time-ordered data. Grp: group splits by the matching rule; Grp, col.: group findings that name the true entity column. Dup.: duplicated raw records. Clean: leak-free files with any finding. LeakLens: first look, result of record.

Tool	**Col.**	**T1** /8	**Grp** /7	**Grp, col.** /7	**Dup.** /2	**Clean** /9
LeakLens (static + run)	no	8	7	3	2	0
leakcheck-ml	no	3	4	4	2	0
	yes	7	7	7	2	0
LeakHound	no	0	0	0	2	0
	yes	7	7	7	2	0
dataleaks	no	0	7	4	2	2
	yes	0	7	7	2	2
Deepchecks	no	0	0	0	2	0
	yes	7	0	0	2	0
splitguard	no	0	0	0	0	0
	yes	0	7	7	0	0

Table 6. Impact estimates against the true inflation, injected TEST, single-leak files, seed 0. N: findings. Comp.: findings whose pipeline reports AUC or R^2, the metrics LeakHound reports. Cells: number of estimates (mean absolute error). LH: LeakHound with its defaults; LH, col.: with the true column named. LeakLens: first look.

Type	**N**	**Comp.**	**LeakLens**	**LH**	**LH, col.**
group	6	6	6 (0.158)	0 (–)	6 (0.075)
overlap	6	6	6 (0.000)	2 (0.043)	2 (0.043)
preproc.	8	8	8 (0.000)	0 (–)	0 (–)
target-der.	3	3	3 (0.000)	0 (–)	0 (–)
temporal	6	3	3 (0.000)	0 (–)	3 (0.059)

LeakHound (Table 6) trains its own model on the exported split and compares it with a group-aware or chronological split it builds; it reports AUC or R^2, so only pipelines with those metrics are compared, on seed 0. Where both tools gave an estimate (11 findings), LeakHound's mean absolute error was 0.065 and LeakLens's 0.086: LeakLens was worse on the hold-out group splits, where it grouped by the wrong column, and exact on duplicated records and time splits, where LeakHound was off by 0.01 to 0.08. With the column fault repaired (second look, not held out) LeakLens's error on the same 11 findings was 0.020. LeakHound gave no estimate for preprocessing or target-derived findings, and none without a named column except for duplicated records. The near-zero preprocessing errors carry little information, as the true inflation of those leaks is close to zero.

6.6 Running time and a negative case study

The static pass took a median of 2.6 ms per file and the instrumented run 1.3 s (maximum 12.7 s) on a laptop; the Yang analyser took a median of 6.2 s per file in a cloud CPU job.

The machines differ, so these figures indicate scale only.

We also ran the static pass on a separate EEG seizure-detection codebase by the first author, with three evaluation regimes of known leakage status: random windows, a within-patient split and leave-one-patient-out. The tool reported nothing in its 78 files: correct for the third regime, a miss for the first two. The code splits with hand-written NumPy masks and by position over lists of custom objects, and subject identity is an object attribute; there is no scikit-learn splitter, data frame or column name to model, and the dynamic pass would not see these splits either. No rule was changed after this result.

7. Limitations and Threats to Validity

No result on real code. All figures come from our own injected benchmark; the tool and the benchmark share authors, and TEST uses different pipelines but the same mutation operators as DEV. The results measure generalisation across pipelines, not across the ways people write leaks, and a precision of 1.00 on short synthetic files will not hold on real notebooks. **Small samples.** TEST has 4 target-derived, 6 multi-test and 7 group leaks. **Blindness.** The column names of the TEST pipelines were seen by mistake while rules were written; no TEST code, label or operator was read, but TEST group results are not blind to entity names. **Heuristic entity inference.** Evidence for an entity is a name, repetition or a file path; a name can mislead in both directions, as the first TEST run shows. **Coverage of code styles.** pandas, NumPy and scikit-learn are modelled by name; hand-written splits, custom classes and deep learning frameworks are largely outside the analysis, as the case study shows. **Impact estimates.** Automatic fixes exist for six situations, about half of the located leaks; a group-aware or chronological split changes which rows are evaluated, so an estimate is the change under the canonical fix, not a causal decomposition. **Baselines.** The Yang analyser was scored under a looser rule than the protocol's; the data-level tools were given tables exported by our instrumented run and ran after the single TEST run of LeakLens; scicode-lint and LeakCheck could not be run.

8. Conclusion and Future Work

LeakLens detects six types of data leakage in Python ML code, including group leakage from source code, links static findings to an instrumented run, and estimates a located leak's effect by re-running the pipeline with a fix. On held-out injected leaks it found 81% of leaks statically and 89% with the run, with no false positive, and more leaks than the reference static analyser on the three types they share ($p = 0.001$). Against data-level tools given the same splits it was the only tool to find the time-order leaks without being told the time column, and it was matched or beaten on group leaks by a tool told the entity column. The evaluation also exposed a confirmation fault that named the wrong entity in four of seven group findings, and a case study on real code showed that hand-written splits are invisible to it.

Next, labelling the 150 sampled notebooks will show whether these results hold on real code; we then plan to run scicode-lint on FP8 hardware, cover manual and framework-specific splits, and measure how often each leakage type occurs in public notebooks and how much it changes their scores.

Data and Code Availability

The source code, benchmark generators, frozen protocol, evaluation change log and the output of every run cited here are in the project repository, released under the MIT licence: `https://github.com/bytes06runner/LeakLens`.

Acknowledgement

The implementation was written with the help of an AI coding assistant. The authors made the design and protocol decisions and reviewed the results reported here.

References

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, 2012. doi:10.1145/2382577.2382579.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 100804, 2023. doi:10.1016/j.patter.2023.100804.

[3] C. Yang, R. A. Brower-Sinning, G. A. Lewis, and C. Kästner, "Data leakage in notebooks: static detection and better processes," in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2022. doi:10.1145/3551349.3556918.

[4] S. Roth, "Which leakage types matter? A quantitative landscape across 2,047 benchmark datasets," arXiv:2604.04199, 2026.

[5] G. Brookshire *et al.*, "Data leakage in deep learning studies of translational EEG," *Front. Neurosci.*, vol. 18, 1373515, 2024. doi:10.3389/fnins.2024.1373515.

[6] E. A. AlOmar, C. DeMario, R. Shagawat, and B. Kreiser, "LeakageDetector: an open source data leakage analysis tool in machine learning pipelines," arXiv:2503.14723, 2025.

[7] Truong *et al.*, "LeakageDetector 2.0: analyzing data leakage in Jupyter-driven machine learning pipelines," arXiv:2509.15971, 2025.

[8] T. Draoui, M. W. Mkaouer, and C. Newman, "Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts," *Proc. ACM Softw. Eng.*, vol. 3, no. FSE, 2026. doi:10.1145/3808199.

[9] Samsonau, "scicode-lint: detecting methodology bugs in scientific Python code with LLM-generated patterns," arXiv:2603.17893, 2026.

[10] P. Subotić, L. Milikić, and M. Stojić, "A static analysis framework for data science notebooks," in *Proc. 44th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022. doi:10.1145/3510457.3513032.

[11] P. Subotić, U. Bojanić, and M. Stojić, "Statically detecting data leakages in data science code," in *Proc. 11th ACM SIGPLAN Int. Workshop on the State of the Art in Program Analysis (SOAP)*, 2022. doi:10.1145/3520313.3534657.

[12] F. Drobnjaković, P. Subotić, and C. Urban, "An abstract interpretation-based data leakage static analysis," in *Theoretical Aspects of Software Engineering (TASE)*, LNCS, Springer, 2024. doi:10.1007/978-3-031-64626-3_7.

[13] S. Chorev *et al.*, "Deepchecks: a library for testing and validating machine learning models and data," *J. Mach. Learn. Res.*, vol. 23, no. 285, 2022.

[14] S. Schelter, S. Grafberger, S. Guha, B. Karlaš, and C. Zhang, "Proactively screening machine learning pipelines with ArgusEyes," in *Companion of the 2023 Int. Conf. Management of Data (SIGMOD)*, 2023. doi:10.1145/3555041.3589682.

[15] S. Grafberger, P. Groth, and S. Schelter, "Automating and optimizing data-centric what-if analyses on native machine learning pipelines," *Proc. ACM Manag. Data (SIGMOD)*, 2023. doi:10.1145/3589273.

[16] F. Pedregosa *et al.*, "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[17] T. Aiello, "splitguard," software, version 0.1.0, 2026. `https://github.com/Tommasoaiello13/splitguard`.

[18] J. Gomes, "LeakHound," software, version 0.5.0, 2026. `https://github.com/happyhellpt/leakhound`.

[19] "leakcheck-ml," software, version 0.3.0, 2026. `https://github.com/Sofienguy1/leakcheck`.

[20] K. Rajput, "DataLeaks," software, version 0.1.2, 2026. `https://github.com/KAVYA-29-ai/Dataleaks`.

[21] DeMONLab-BioFINDER, "Leakly: leakage checks for machine-learning pipelines using permutation tests," software, version 0.1.2, 2026. `https://github.com/DeMONLab-BioFINDER/Leakly`.